MoneyTrack AI: full sequence diagram (frontend and backend)

Pages — Login / Register / Dashboard
Components — modals, charts, widgets
api/*.js — axios.js + feature files
Security — SecurityConfig, JwtAuthFilter, JwtUtil
Controllers — Auth, Transaction, Budget...
Services — AuthService, TransactionService...
statement/ — parsers, rules, mapper
Repos + MySQL — JPA repos, tables
Gemini API — external

1. Login (Register is the same path)

1. POST /auth/login
2. public: /api/auth/**
3. AuthController
4. AuthService.login
5. UserRepository.findByUsername
6. user + password hash

BCrypt check. Register also saves the User and EmailService sends the welcome mail

7. JwtUtil.generateToken
8. JWT (24h)
9. AuthResponse(token, username)

token saved in localStorage, go to /dashboard

2. Dashboard loads

10. getTransactions()

axios adds Bearer token

11. GET /api/transactions

JwtAuthFilter verifies token, sets the user

12. TransactionController
13. getAllTransactions
14. TransactionRepository (own rows only)
15. transactions JSON
16. pass data to widgets
17. own api/*.js call
18. Goals, Budgets, Forecast, Charges...
19. services calculate
20. read this user's data
21. JSON for charts, cards, goals

3. Upload a statement

22. uploadStatement(file)
23. POST /api/transactions/upload
24. CsvParserService / PdfParserService
25. List<ParsedTransaction>

locked PDF: 422 PDF_PASSWORD_REQUIRED, Dashboard shows the password box

26. saveParsedTransactions
27. duplicate check, manual match, load rules
28. existing rows + CategoryRules
29. L2: keyword rules
30. L3: Gemini, masked text only
31. category + merchant

validate answer, else fallback to Uncategorized / Unknown

32. TransactionMapper
33. saveAll(transactions)
34. UploadResult (rows, skipped, possibleMatches)
35. dialog + refresh

4. Edit a transaction (the app learns)

36. updateTransaction()
37. PUT /api/transactions/{id}
38. updateTransaction
39. save row + CategoryRuleService.saveRule
40. updated row
41. GET /{id}/similar, POST /{id}/apply-to-similar

rows with the same description get the same category and merchant

42. LearnedRules: GET / DELETE /api/rules

5. AI and planning features

43. tips, Montra, leftover split (via api/*.js)
44. call service

PersonalTipService, QuestionIntentService + AssistantService, LeftoverSplitService

45. SpendingProfileService + AiCacheService
46. profile + cached answer
47. only if not cached
48. tips / intent / split as JSON

validate the JSON, compute the real numbers in code

49. save to ai_cache
50. tips, answer, split

6. Logout

handleLogout: remove token, go to /login. The backend is not involved.

request — response — note: work done inside that layer

Purple = frontend (src/), green = backend (com.expensetracker.backend), gray = database and external AI. Every call after login carries the JWT, and every query is limited to that user.